

Universidade Federal de Ouro Preto - UFOP
Instituto de Ciências Exatas e Biológicas - ICEB
Departamento de Computação - DECOM
Ciência da Computação

Trabalho Prático III

PCC104 - Projeto e Análise de Algoritmos

Fernando dos Santos Alves Fernandes
Professor: Pedro Henrique Lopes Silva

Ouro Preto
15 de julho de 2024

Sumário

1	Introdução	1
1.1	Algoritmos Aproximados	1
1.1.1	Hill-climbing	1
1.1.2	Simulated Annealing	2
1.1.3	Tabu search	2
1.1.4	Genetic Algorithm	3
1.2	Visão geral do ambiente de desenvolvimento e testes	3
1.2.1	Ambiente de desenvolvimento e ferramentas utilizadas	4
1.2.2	Instruções de compilação e execução	4
2	Implementação	4
2.1	Implementações para o Problema do Clique Máximo	4
2.2	Implementações para o Problema do Mínimo Número de Cliques	11
3	Estudo de Complexidade	18
3.1	Análise de complexidade para o Problema do Clique Máximo	18
3.2	Análise de complexidade para o Problema do Mínimo Número de Cliques	19
4	Testes	19
4.1	Testes para o Problema do Clique Máximo	19
4.2	Testes para o Problema do Mínimo Número de Cliques	21
5	Análises e Considerações	27

Lista de Tabelas

1	Exemplo de entrada e saída esperada para a Tarefa 1	20
2	Exemplo de entrada e saída para a Tarefa 2	22
3	Resultados dos testes: Comparativo entre estratégias.	28

Lista de Códigos Fonte

1	Pseudo-código - Hill-climbing genérico. Adaptado de [4]	1
2	Pseudo-código - Simulated Annealing genérico. Adaptado de [4]	2
3	Pseudo-código - Busca Tabu genérica. Adaptado de [4]	2
4	Pseudo-código - Genetic Algorithm genérico. Adaptado de [4]	3
5	Implementação da Tarefa 1 - Problema do Clique Máximo, usando <i>Hill-climbing</i> , <i>Simulated Annealing</i> e <i>Tabu Search</i>	5
6	Implementação da Tarefa 2 - Problema do Mínimo Número de Cliques, com <i>Hill-climbing</i> , <i>Simulated Annealing</i> e <i>Tabu Search</i>	11

1 Introdução

Neste terceiro trabalho prático da disciplina de Projeto e Análise de Algoritmos, as tarefas estão relacionadas a lidar com dois problemas computacionais combinatórios: obter o **maior clique dentro de um grafo não-direcionado** e obter o **menor número de cliques dentro de um grafo não-direcionado**. No entanto, diferente do trabalho prático anterior, neste caso o interesse é na obtenção de soluções aproximadas. Portanto, as **Tarefa 1** e **Tarefa 2** consistiram em implementar algoritmos aproximados para resolver os dois problemas.

Um clique em um grafo não-direcionado $G = (V, E)$ é um subconjunto $V' \subseteq V$ de vértices, em que cada par $\{u, v\}$, com $u \in V'$ e $v \in V'$, é uma aresta em E . Em outras palavras, um clique induz um subgrafo completo em G . O tamanho do clique é o número de vértices que ele contém. O **problema do clique máximo** é um problema de otimização, cujo objetivo é encontrar o clique de tamanho máximo em um grafo G [2].

O problema dos subconjuntos consiste na minimização do número de cliques dentro de um grafo não-direcionado. O problema de encontrar o número mínimo de cliques em um grafo G está intimamente relacionado ao problema de particionar os vértices de G em subconjuntos, em que cada subconjunto forma um clique. Esse problema é também conhecido como o problema da **cobertura mínima por cliques** (*minimum covering by cliques*) [3].

1.1 Algoritmos Aproximados

Dentre as possíveis abordagens para resolver os dois problemas mencionados, encontram-se os algoritmos de busca exaustiva com "poda", como o **Backtracking** e o **Branch&Bound**, que foram objeto de estudo no TP II. Embora as podas diminuam o espaço de busca de ambos algoritmos, essas estratégias ainda demandam muito tempo de execução. Para problemas dessa natureza, estratégias aproximadas ou *algoritmos aproximados* podem ser úteis na obtenção de soluções aproximadas ou subótimas.

Os **algoritmos aproximados**, também chamados de *algoritmos heurísticos* são algoritmos que tentam resolver problemas de otimização por meio de heurísticas, ou seja, uso de uma sequência de pequenas alterações em uma dada solução parcial, de forma a obter uma solução diferente, possivelmente mais próxima da solução ótima. Essas alterações, também chamadas de *movimentos* ou *perturbações* são realizadas com base no conceito de busca pela vizinhança (ou *neighborhood search*) e um algoritmo heurístico opera realizando, iterativamente, uma ou mais heurísticas, conforme a estratégia.

Dentre as principais estratégias para o projeto de **algoritmos aproximados** encontram-se: *Hill-climbing*, *Simulated Annealing*, *Tabu search* e *Genetic Algorithms*, brevemente descritas a seguir.

1.1.1 Hill-climbing

A estratégia *Hill-climbing* é uma das estratégias mais simples. A ideia de escalar montanhas ou subir a colina sugere que a forma mais rápida de alcançar o topo é subir continuamente. O que se procura com essa ideia é que, na busca pela vizinhança, *perturbações* dão preferência por soluções viáveis cujo "lucro" é maior do que o da solução atual [4].

Inicia-se de um ponto aleatório X , ou seja, com uma solução viável qualquer, e faz-se uma avaliação dessa solução. Move-se para um novo ponto vizinho Y , que corresponde a uma nova solução viável gerada a partir de X . Uma sequência desses pequenos movimentos gera uma solução melhor [1].

Assim, encontrar uma sequência de soluções viáveis, cada uma das quais melhor do que a anterior pode levar em direção a uma *solução ótima local* [4].

A seguir, uma versão genérica do algoritmo (Pseudo-código 1).

```
1  GENERIC_HILL_CLIMBING(cmax)
2  external: N(), hN(), P()
3  comment: if hN returns a feasible solution Y, then P(Y) > P(X)
4  1 Select a feasible X in U (universo de solucoes)
5  2 Xbest <- X
6  3 searching <- true
7  4 while searching do
8      5     Y <- hN(X)
9      6     if Y != Fail then
10         7         Y <- X
11         8         if P(X) > P(Xbest) then
```

```

12 9         Xbest <- X
13 10     else searching <- false
14 11 return (Xbest)

```

Código 1: Pseudo-código - Hill-climbing genérico. Adaptado de [4]

1.1.2 Simulated Annealing

Uma estratégia alternativa que procura evitar a limitação de encontrar *soluções ótimas locais* é baseada numa analogia com um método de resfriamento de metal conhecida como “annealing” (ou recozimento), por isso é conhecida como *Simulated Annealing* ou “recozimento simulado”. De forma similar ao *Hill-climbing*, inicia-se com uma solução viável aleatória X e faz-se sua avaliação. Move-se para uma solução vizinha Y , gerada por uma perturbação em X . Entretanto, ao contrário do *Hill-climbing*, que interrompe a busca quando não encontra mais uma solução vizinha com lucro maior, o *Simulated Annealing* considera soluções “inferiores”, segundo uma probabilidade [1].

Como outras estratégias, a busca pela vizinhança é realizada de forma aleatória. Se uma solução viável é encontrada e seu lucro é maior ou igual ao da solução corrente, essa solução da vizinhança é considerada melhor. No entanto, às vezes (de forma probabilística) o método considera uma solução viável vizinha mesmo que o lucro não seja melhor do que o da solução atual. A variável *temperatura* (T) é diminuída de acordo com uma taxa (*cooling schedule*), associada à probabilidade de se considerar soluções viáveis menos “lucrativas”. Isso ajuda a evitar que o algoritmo fique restrito a soluções ótimas locais [4].

A seguir, uma versão genérica do psudo-código para o algoritmo (Pseudo-código 2).

```

1  GENERIC_SIMULATED_ANNEALING(cmax, T0, alpha)
2  external: N(), hN(), Random(), P()
3  1 c <- 0
4  2 T <- T0
5  3 Select a feasible X in U (universo de solucoes)
6  4 Xbest <- X
7  5 while c <= cmax
8  6     Y <- hN(X)
9  7     if Y != Fail then
10 8         if P(Y) > P(X) then
11 9             X <- X
12 10            if P(X) > P(Xbest) then
13 11                Xbest <- X
14 12            else
15 13                r <- Random(0,1)
16 14                if r < e^((P(Y) - P(X))/T) then
17 15                    X <- Y
18 16            c <- c + 1
19 17            T <- alpha * T
20 18 return (Xbest)

```

Código 2: Pseudo-código - Simulated Annealing genérico. Adaptado de [4]

1.1.3 Tabu search

A Busca Tabu ou *Tabu search* tem como ideia básica substituir um elemento X por um elemento Y da vizinhança de X , tal que Y é viável e o lucro de Y é o máximo entre todas as soluções viáveis. Para garantir isso, a função de busca pela vizinhança de X é uma busca exaustiva e uma *lista tabu* é utilizada para evitar que o algoritmo entre num *loop* infinito, uma vez que pode ficar realizando trocas infinitamente entre as soluções vizinhas X e Y . Por um tempo predefinido, uma troca entre soluções vizinhas é proibida e a *lista tabu* é que faz esse controle. A versão genérica desse algoritmo é apresentada em Pseudo-código 3 e mais detalhes sobre essa estratégia podem ser obtidos em [4].

```

1  GENERIC_TABU_SEARCH(cmax, L)
2  external: N(), change(), P()
3  1 c <- 1
4  2 Select a feasible X in U (universo de solucoes)

```

```

5 3 Xbest <- X
6 4 while c <= cmax
7 5     N <- N(X) \ {TabuList[d]: c - L <= d <= c - 1}
8 6     for each Y in N do
9 7         if Y is infeasible then
10 8             N <- N \ {Y}
11 9         if N is empty then
12 10             exit
13 11     find Y in N such that P(Y) is maximum
14 12     TabuList[c] <- change(Y, X)
15 13     X <- Y
16 14     if P(X) > P(Xbest) then
17 15         Xbest <- X
18 16     c <- c + 1
19 17 return (Xbest)

```

Código 3: Pseudo-código - Busca Tabu genérica. Adaptado de [4]

1.1.4 Genetic Algorithm

Nas estratégias anteriores, *hill-climbing*, *simulated annealing* e *tabu search*, a ideia de busca pela vizinhança é utilizada em uma solução viável inicial para construir uma solução subótima por meio da aplicação de uma heurística em uma sequência de soluções viáveis. No algoritmo genético (ou *Genetic Algorithm*), inicia-se com uma população inicial (*initial population*) de soluções viáveis. Realiza-se um “casamento” ou um cruzamento (*crossover*) entre indivíduos dessa população inicial para produzir “filhos”. Esses filhos podem sofrer mutações (*mutation*), que podem ser heurísticas baseadas em busca pela vizinhança, gerando uma próxima geração. E esse processo itera tanto quanto o número desejado de gerações.

```

1 GENERIC_GENETIC_ALGORITHM(popsize, cmax)
2 external: N(), hN(), rec(), P()
3 1 c <- 1
4 2 Select an initial population P of popsize feasible solutions
5 3 Let Xbest be the element in P having maximum profit
6 4 for each X in P do
7 5     X <- hN(X)
8 6 while c <= cmax do
9 7     Construct a pairing of the elements in P
10 8     Q <- P
11 9     for each pair W, X in pairing do
12 10         (Y,Z) <- rec(W,X)
13 11         Y <- hN(Y)
14 12         Z <- hN(Z)
15 13         Q <- Q U {Y, Z}
16 14 Let the population P the best popsize members of Q
17 15 Let Y be the element in P having maximum profit
18 16 if P(Y) > P(Xbest) then
19 17     Xbest <- Y
20 18 c <- c + 1
21 19 return (Xbest)

```

Código 4: Pseudo-código - Genetic Algorithm genérico. Adaptado de [4]

1.2 Visão geral do ambiente de desenvolvimento e testes

A seguir, uma visão geral do ambiente de desenvolvimento e testes, com instruções de compilação e uso das soluções implementadas.

1.2.1 Ambiente de desenvolvimento e ferramentas utilizadas

Os algoritmos desenvolvidos para a solução das duas tarefas foram implementados em linguagem *C++* e compilados usando o *gcc 6.3.0* do conjunto de ferramentas MinGW¹. Os programas gerados foram executados em ambiente *Windows 10 Pro - 64 bits*, numa máquina *Intel(R) Core(TM) i5-7500 CPU @ 3.40 GHz 3.41 GHz*, com *16 GB RAM*.

Os programas também foram compilados com o *g++ 5.40 20160609* e executados em ambiente *Linux (Ubuntu 16.04.12 LTS)*, numa máquina *Intel(R) Core(TM) i7-3770 CPU @ 3.40 GHz x8*, com *16 GB RAM*².

1.2.2 Instruções de compilação e execução

Para a compilação do código associado à Tarefa 1, basta digitar, no terminal:

```
Compilando código para o problema do clique máximo
```

```
gcc task1-max-clique.cpp -o task1-max-clique
```

Para a execução do programa basta digitar:

```
./task1-max-clique.exe <arquivo.clq | arquivo.grafo> <maxtime> <algoritmo [hc: hill-climbing  
| sa: simulated annealing | ts: tabu search] >
```

Para a compilação do código associado à Tarefa 2, basta digitar, no terminal:

```
Compilando código para o problema do mínimo número de cliques
```

```
gcc task2-min-cliques.cpp -o task2-min-cliques
```

Para a execução do programa basta digitar:

```
./task2-min-cliques.exe <arquivo.clq | arquivo.grafo> <maxtime> <algoritmo [hc: hill-  
climbing | sa: simulated annealing | ts: tabu search] >
```

2 Implementação

Nesta seção serão apresentadas as soluções implementadas para o *problema do máximo clique* e para o *problema do mínimo número de cliques*. Para cada solução, descreve-se o funcionamento das principais funções e procedimentos utilizados, bem como detalhes de implementação.

2.1 Implementações para o Problema do Clique Máximo

Para o *problema do máximo clique* foram implementadas três das estratégias de projeto de algoritmos aproximados: *Hill-climbing*, *Simulated Annealing* e *Tabu Search* (Código (5)).

Para a estratégia *Hill-climbing*, a função `vector<int> hillClimbing()` implementa todas as etapas que caracterizam a *subida de encosta*. O algoritmo começa com uma solução inicial (*estado inicial*), obtida aleatoriamente (`vector<int> getRandomClique()`); na sequência, realiza-se uma busca pela vizinha (`vector<int> getNeighbors(...)`) e a seleção do melhor vizinho (linha 92 do código), “movendo-se” para esse estado; o processo se repete até que não haja vizinhos que melhorem a solução atual ou até que um dos critérios de parada tenha sido atingido (limite máximo de tempo de execução).

No *Simulated Annealing*, de forma similar ao *Hill-climbing*, o algoritmo define uma solução inicial de forma aleatória. No entanto, esta estratégia trabalha com os parâmetros de temperatura inicial (*initialTemperature*) e taxa de resfriamento (*coolingRate*): `vector<int> simulatedAnnealing(double initialTemperature, double coolingRate)`.

¹<https://sourceforge.net/projects/mingw/files/MinGW/Base/gcc/Version6/gcc-6.3.0/>

²Arquivo de instruções: em <https://acesse.dev/Shucm>

Enquanto a temperatura é maior do que o limite mínimo, gera-se uma nova solução vizinha e calcula-se a diferença de energia (ou custo) entre a nova solução e a solução atual. Se a nova solução for melhor, passa a ser a solução corrente.

Se a solução vizinha não for melhor, ainda assim ela pode ser aceita, com uma probabilidade (*double probability(double delta, double temperature)*) que depende da diferença de energia e da temperatura. Essa função diminui conforme a temperatura cai, fazendo com que seja menos provável aceitar soluções piores conforme o algoritmo avança.

O processo se repete e a temperatura é diminuída de acordo com a taxa de resfriamento até que seja menor do 1. O resultado é o clique com o maior número de vértices encontrado até esse momento.

Outra estratégia de projeto de algoritmos aproximados implementada neste trabalho foi a Busca Tabu ou *Tabu Search* (*vector<int> tabuSearch(int maxIterations, int maxTabuSize)*), que se caracteriza por manter uma lista (*lista tabu*) que controla se um vértice pode ou não ser adicionado à uma solução vizinha.

Como as estratégias anteriores a Busca Tabu começa com uma solução aleatória, verifica as soluções vizinhas, mas só as considera se o vértice a ser adicionado não estiver na *lista tabu*. Se o tamanho máximo definido para essa lista for atingido, o vértice mais antigo é removido e o mais recente é adicionado à lista. O processo se repete até que o número máximo de iterações definido (ou o tempo máximo de execução) seja atingido.

```

1  /**
2   * PCC104 - Projeto e Analise de Algoritmos
3   * Departamento de Computacao - Universidade Federal de Ouro Preto - MG
4   * Professor: Pedro Silva
5   * Aluno: Fernando dos Santos Alves Fernandes
6   * Trabalho Pratico III (TP III) - Tarefa 1: Problema do Clique Maximo
7   * Algoritmos: Hill Climbing, Simulated Annealing e Tabu Search
8   */
9
10 #include <iostream>
11 #include <fstream>
12 #include <sstream>
13 #include <vector>
14 #include <string>
15 #include <algorithm>
16 #include <chrono>
17 #include <bits/stdc++.h> //setprecision
18 #include <ctime>
19 #include <cstdlib>
20 #include <cmath>
21 #include <set>
22 #include <numeric>
23 #include <unordered_set>
24
25 using namespace std;
26
27 class Graph {
28 private:
29     int V;
30     vector<vector<int> > adj;
31     double maxTime;
32     chrono::time_point<chrono::_V2::system_clock, chrono::duration<long long
33         int, ratio<111, 100000000011> > > initTime;
34     chrono::time_point<chrono::_V2::system_clock, chrono::duration<long long
35         int, ratio<111, 100000000011> > > endTime;
36     chrono::duration<long long int, ratio<111, 100000000011> > elapsedTime;
37
38 public:
39     Graph(int V) : V(V) {
40         adj.resize(V);

```

```

41 void setInitialAndMaxTime(chrono::time_point<chrono::_V2::system_clock,
    chrono::duration<long long int, ratio<11l, 100000000001l> > > initial,
    double time) {
42     initTime = initial;
43     maxTime = time;
44 }
45
46 auto getElasedTime(){
47     return elapsedTime.count();
48 }
49
50 void printElapsedTime(){
51     cout << "Tempo de execucao: " << fixed << setprecision(4) << double (
        elapsedTime.count() * 1e-9) << " s."<< endl;
52 }
53
54 void addEdge(int u, int v) {
55     adj[u].push_back(v);
56     adj[v].push_back(u);
57 }
58
59 bool isClique(const vector<int>& vertices) { // Avaliacao se os vertices
    formam um clique
60     for (size_t i = 0; i < vertices.size(); ++i) {
61         for (size_t j = i + 1; j < vertices.size(); ++j) {
62             if (find(adj[vertices[i]].begin(), adj[vertices[i]].end(),
                vertices[j]) == adj[vertices[i]].end()) {
63                 return false;
64             }
65         }
66     }
67     return true;
68 }
69
70 vector<int> getRandomClique() { // Estado inicial
71     vector<int> clique;
72     int start = rand() % V;
73     clique.push_back(start);
74     return clique;
75 }
76
77 vector<int> getNeighbors(const vector<int>& clique) { // Busca pela
    vizinhanca
78     vector<int> neighbors;
79     for (int v : clique) {
80         for (int u : adj[v]) {
81             if (find(clique.begin(), clique.end(), u) == clique.end()) {
82                 neighbors.push_back(u);
83             }
84         }
85     }
86     return neighbors;
87 }
88
89 vector<int> hillClimbing() {
90     // Verifica quanto tempo decorrido
91     endTime = chrono::high_resolution_clock::now();
92     auto elapsed = chrono::duration_cast<chrono::nanoseconds>(endTime -
        initTime);
93
94     // Inicializa uma clique aleatoria
95     vector<int> current = getRandomClique();

```

```

96
97 // Loop principal que continua ate que nao se possa encontrar um
    clique maior ou o tempo maximo seja atingido
98 while (true && (elapsed < chrono::duration_cast<chrono::nanoseconds>(
    chrono::duration<double>(maxTime)))) {
99     // Obtem os vizinhos da clique atual
100     vector<int> neighbors = getNeighbors(current);
101     // Define a melhor clique vizinha como a clique atual
102     vector<int> bestNeighbor = current;
103
104     // Itera sobre os vizinhos
105     for (int neighbor : neighbors) {
106         // Cria um novo clique adicionando o vizinho na clique atual
107         vector<int> newClique = current;
108         newClique.push_back(neighbor);
109
110         // Verifica se o novo conjunto e uma clique e se e maior que a
            melhor vizinha atual
111         if (isClique(newClique) && newClique.size() > bestNeighbor.
            size()) {
112             bestNeighbor = newClique;
113         }
114     }
115
116     // Se a melhor vizinha tem o mesmo tamanho da clique atual,
        interrompe o loop
117     if (bestNeighbor.size() == current.size()) {
118         break;
119     }
120     // Atualiza a clique atual para a melhor vizinha encontrada
121     current = bestNeighbor;
122
123     // Atualiza o tempo decorrido
124     endTime = chrono::high_resolution_clock::now();
125     auto elapsed = chrono::duration_cast<chrono::nanoseconds>(endTime
        - initTime);
126     elapsedTime = elapsed;
127 }
128 // Retorna a clique atual, que e a maior clique encontrada
129 return current;
130 }
131
132 // Funcao para calcular a probabilidade de aceitacao de uma solucao pior
133 double probability(double delta, double temperature) {
134     return exp(delta / temperature);
135 }
136
137 vector<int> simulatedAnnealing(double initialTemperature, double
    coolingRate) {
138     // Verifica quanto tempo decorrido
139     endTime = chrono::high_resolution_clock::now();
140     auto elapsed = chrono::duration_cast<chrono::nanoseconds>(endTime -
        initTime);
141
142     // Inicializa uma clique aleatoria e define o melhor clique inicial
        como esta clique
143     vector<int> current = getRandomClique();
144     vector<int> best = current;
145     // Define a temperatura inicial
146     double temperature = initialTemperature;
147
148     // Loop principal que continua ate a temperatura cair abaixo de 1 ou o

```

```

149         tempo maximo ser atingido
150     while (temperature > 1 && (elapsed < chrono::duration_cast<chrono::
151         nanoseconds>(chrono::duration<double>(maxTime)))) {
152         // Obtem os vizinhos da clique atual
153         vector<int> neighbors = getNeighbors(current);
154         // Se nao houver vizinhos, interrompe o loop
155         if (neighbors.empty()) {
156             break;
157         }
158         // Seleciona um vizinho aleatorio
159         int idx = rand() % neighbors.size();
160         vector<int> newClique = current;
161         newClique.push_back(neighbors[idx]);
162
163         // Verifica se o novo conjunto e uma clique
164         if (isClique(newClique)) {
165             // Aceita a nova clique se for maior que a clique atual ou com
166             // uma certa probabilidade mesmo que seja for menor
167             if (newClique.size() > current.size() || probability(newClique
168                 .size() - current.size(), temperature) > (double)rand() /
169                 RAND_MAX) {
170                 current = newClique;
171             }
172             // Atualiza o melhor clique encontrado ate agora
173             if (current.size() > best.size()) {
174                 best = current;
175             }
176         }
177         // Reduz a temperatura
178         temperature *= coolingRate;
179
180         // Atualiza o tempo decorrido
181         endTime = chrono::high_resolution_clock::now();
182         auto elapsed = chrono::duration_cast<chrono::nanoseconds>(endTime
183             - initTime);
184         elapsedTime = elapsed;
185     }
186     // Retorna a melhor clique encontrada
187     return best;
188 }
189
190 vector<int> tabuSearch(int maxIterations, int maxTabuSize) {
191     // Verifica quanto tempo decorrido
192     endTime = chrono::high_resolution_clock::now();
193     auto elapsed = chrono::duration_cast<chrono::nanoseconds>(endTime -
194         initTime);
195
196     // Inicializa uma clique aleatoria e define o melhor clique inicial
197     // como esta clique
198     vector<int> current = getRandomClique();
199     vector<int> best = current;
200
201     // Define uma lista tabu para evitar ciclos
202     unordered_set<int> tabuList;
203     int iterations = 0;
204
205     // Loop principal que continua ate o maximo de iteracoes ou tempo
206     // maximo
207     while (iterations < maxIterations && (elapsed < chrono::duration_cast<
208         chrono::nanoseconds>(chrono::duration<double>(maxTime)))) {
209         // Obtem os vizinhos da clique atual
210         vector<int> neighbors = getNeighbors(current);

```

```

201         // Se nao houver vizinhos, interrompe o loop
202         if (neighbors.empty()) {
203             break;
204         }
205
206         // Variavel para armazenar o melhor vizinho encontrado
207         vector<int> bestNeighbor;
208         // Itera sobre os vizinhos
209         for (int neighbor : neighbors) {
210             // Se o vizinho nao esta na lista tabu
211             if (tabuList.find(neighbor) == tabuList.end()) {
212                 // Cria um novo clique adicionando o vizinho na clique
213                 // atual
214                 vector<int> newClique = current;
215                 newClique.push_back(neighbor);
216
217                 // Verifica se o novo conjunto e uma clique e se e maior
218                 // que o melhor vizinho atual
219                 if (isClique(newClique) && newClique.size() > bestNeighbor
220                     .size()) {
221                     bestNeighbor = newClique;
222                 }
223             }
224         }
225
226         // Se nenhum vizinho valido foi encontrado, interrompe o loop
227         if (bestNeighbor.empty()) {
228             break;
229         }
230
231         // Atualiza a clique atual para o melhor vizinho encontrado
232         current = bestNeighbor;
233
234         // Se a clique atual e maior que a melhor encontrada ate agora,
235         // atualiza a melhor clique e limpa a lista tabu
236         if (current.size() > best.size()) {
237             best = current;
238             tabuList.clear();
239         }
240
241         // Adiciona o ultimo vertice da clique atual na lista tabu
242         tabuList.insert(current.back());
243         // Se a lista tabu excede o tamanho maximo permitido, remove o
244         // elemento mais antigo
245         if (tabuList.size() > maxTabuSize) {
246             tabuList.erase(tabuList.begin());
247         }
248
249         // Incrementa o contador de iteracoes e atualiza o tempo decorrido
250         iterations++;
251
252         // Contabiliza o tempo de execucao ate o momento
253         endTime = chrono::high_resolution_clock::now();
254         auto elapsed = chrono::duration_cast<chrono::nanoseconds>(endTime
255             - initTime);
256         elapsedTime = elapsed;
257     }
258     // Retorna a melhor clique encontrada
259     return best;
260 }
261 };

```

```

257 int main(int argc, char* argv[]) {
258     if (argc != 4) {
259         cerr << "Uso: " << argv[0] << " <arquivoProblema> <tempoMaximo> <
                strategy>" << endl;
260         return 1;
261     }
262
263     string filename = argv[1];
264     int timetorun = stoi(argv[2]);
265     string strategy = argv[3];
266
267     ifstream file(filename);
268     if (!file.is_open()) {
269         cerr << "Erro ao abrir o arquivo " << filename << endl;
270         return 1;
271     }
272
273     string line;
274     unsigned int numVertices = 0;
275     unsigned int numEdges = 0;
276
277     string extension = ".grafo";
278     string::size_type i = filename.find(extension);
279     if (i != string::npos) { // extensao '.grafo'
280         getline(file, line);
281         stringstream ss(line);
282         ss >> numVertices;
283         ss >> numEdges;
284     } else { // extensao '.clq'
285         // Ler o cabeçalho do arquivo e obter o numero de vertices e arestas
286         while (getline(file, line)) {
287             if (line.empty() || line[0] == 'p') {
288                 stringstream ss(line);
289                 string token;
290                 ss >> token; // descartar 'p'
291                 ss >> token; // esperado 'col'/'edge'
292                 ss >> numVertices;
293                 ss >> numEdges;
294                 break;
295             }
296         }
297     }
298     srand(time(0));
299
300     Graph graph(numVertices);
301
302     // Ler as arestas e adicionar ao grafo
303     if (i != string::npos) { // extensao '.grafo'
304         while (getline(file, line)) {
305             if (line.empty()) continue;
306             unsigned int from, to;
307             stringstream ss(line);
308             ss >> from >> to;
309             graph.addEdge(from, to); // Os vertices no arquivo comecam de 0.
310         }
311     } else {
312         while (getline(file, line)) {
313             if (line.empty() || line[0] != 'e') continue;
314             stringstream ss(line);
315             string token;
316             ss >> token; // descartar 'e'
317             unsigned int from, to;

```

```

318         ss >> from >> to;
319         graph.addEdge(from - 1, to - 1); // Os vertices no arquivo comecam
           de 1.
320     }
321 }
322 file.close();
323
324 srand(time(0));
325
326 auto initTime = chrono::high_resolution_clock::now();
327 graph.setInitialAndMaxTime(initTime, timetorun);
328
329 vector<int> clique;
330 if (strategy == "hc") // Hill-climbing
331     clique = graph.hillClimbing();
332 else if (strategy == "sa") // Simulated Annealing
333     clique = graph.simulatedAnnealing(1000, 0.95); // Temperatura inicial
           e temperatura minima
334 else if (strategy == "ts") // Tabu Search
335     clique = graph.tabuSearch(10000, 7); // Maximo de iteracoes e tamanho
           maximo da lista tabu
336 graph.printElapsedTime();
337
338 cout << clique.size() << endl;
339 for (int v : clique) {
340     cout << v + 1 << " ";
341 }
342 cout << endl;
343 return 0;
344 }

```

Código 5: Implementação da Tarefa 1 - Problema do Clique Máximo, usando *Hill-climbing*, *Simulated Annealing* e *Tabu Search*.

O usuário seleciona a estratégia desejada via linha de comando ('hc' = *Hill-climbing*, 'sa' = *Simulated Annealing* ou 'ts' = *Tabu Search*), conforme as instruções de compilação e execução na Seção 1.2.2.

2.2 Implementações para o Problema do Mínimo Número de Cliques

O Código 6 apresenta a implementação para resolver o problema de encontrar o **mínimo número de cliques** em um grafo não direcionado, utilizando como estratégias *Hill-climbing*, *Simulated Annealing* e *Tabu Search*.

Para essa tarefa foi implementada a função `vector<vector<int>> vertexCoverByCliques (int timeLimit, string strategy)`, que chama a função correspondente à estratégia desejada: `hillClimbing (coveredVertices)`, `simulatedAnnealing (coveredVertices, maxIterations, initialTemp, coolingRate)` ou `tabuSearch (coveredVertices, maxIterations, tabuTenure)`.

Cada estratégia busca encontrar a cobertura mínima por cliques, dentro do limite de tempo (`timeLimit`) e/ou número máximo de iterações (`maxIterations`). Inicialmente, os valores tomados para `initialTemp`, `coolingRate` e `tabuTenure` foram, respectivamente, 1000.0, 0.95 e 10.

```

1 /**
2  * PCC104 - Projeto e Analise de Algoritmos
3  * Departamento de Computacao - Universidade Federal de Ouro Preto - MG
4  * Professor: Pedro Silva
5  * Aluno: Fernando dos Santos Alves Fernandes
6  * Trabalho Pratico III (TP III) - Tarefa 2: Problema do Minimo Numero de
   Cliques
7  * Algoritmos aproximados: Hill-climbing, Simulated Annealing, Tabu Search
8  */
9
10 #include <iostream>

```

```

11 #include <fstream>
12 #include <sstream>
13 #include <vector>
14 #include <string>
15 #include <algorithm>
16 #include <chrono>
17 #include <bits/stdc++.h> //setprecision
18 #include <ctime>
19 #include <cstdlib>
20 #include <set>
21 #include <numeric>
22 #include <unordered_set>
23 #include <unordered_map>
24
25 using namespace std;
26
27 class Graph {
28 public:
29     int V;
30     vector<vector<int>> > adj;
31     double maxTime;
32     chrono::time_point<chrono::_V2::system_clock, chrono::duration<long long
        int, ratio<111, 100000000011> > > > initTime;
33     chrono::time_point<chrono::_V2::system_clock, chrono::duration<long long
        int, ratio<111, 100000000011> > > > endTime;
34     chrono::duration<long long int, ratio<111, 100000000011> > > elapsedTime;
35
36     Graph(int V) : V(V), adj(V) {}
37
38     void addEdge(int u, int v) {
39         adj[u].push_back(v);
40         adj[v].push_back(u);
41     }
42
43     bool isClique(const vector<int>& vertices) {
44         for (size_t i = 0; i < vertices.size(); ++i) {
45             for (size_t j = i + 1; j < vertices.size(); ++j) {
46                 if (find(adj[vertices[i]].begin(), adj[vertices[i]].end(),
47                     vertices[j]) == adj[vertices[i]].end()) {
48                     return false;
49                 }
50             }
51         }
52         return true;
53     }
54
55     vector<int> getRandomClique(const unordered_set<int>& coveredVertices) {
56         vector<int> clique;
57         int start;
58         do {
59             start = rand() % V;
60         } while (coveredVertices.find(start) != coveredVertices.end());
61         clique.push_back(start);
62         return clique;
63     }
64
65     vector<int> getNeighbors(const vector<int>& clique, const unordered_set<
        int>& coveredVertices) {
66         vector<int> neighbors;
67         for (int v : clique) {
68             for (int u : adj[v]) {
69                 if (find(clique.begin(), clique.end(), u) == clique.end() &&

```

```

69         coveredVertices.find(u) == coveredVertices.end()) {
70             neighbors.push_back(u);
71         }
72     }
73     return neighbors;
74 }
75
76 vector<int> hillClimbing(const unordered_set<int>& coveredVertices) {
77     // Inicializa a clique atual com um vertice aleatorio nao coberto
78     vector<int> current = getRandomClique(coveredVertices);
79
80     // Loop principal
81     while (true) {
82         // Obtem os vizinhos da clique atual que nao estao cobertos
83         vector<int> neighbors = getNeighbors(current, coveredVertices);
84
85         // Inicializa a melhor vizinhanca como a clique atual
86         vector<int> bestNeighbor = current;
87
88         // Itera sobre todos os vizinhos possiveis
89         for (int neighbor : neighbors) {
90             // Cria uma nova clique adicionando o vizinho ao atual
91             vector<int> newClique = current;
92             newClique.push_back(neighbor);
93
94             // Verifica se a nova clique e de fato uma clique e se e maior
95             // que a melhor vizinhanca atual
96             if (isClique(newClique) && newClique.size() > bestNeighbor.
97                 size()) {
98                 bestNeighbor = newClique; // Atualiza a melhor vizinhanca
99                 encontrada
100             }
101         }
102
103         // Se nao houve melhoria na clique atual, termina o algoritmo
104         if (bestNeighbor.size() == current.size()) {
105             break;
106         }
107
108         // Atualiza a clique atual para a melhor vizinhanca encontrada
109         current = bestNeighbor;
110     }
111
112     // Retorna a melhor clique encontrada
113     return current;
114 }
115
116 vector<int> simulatedAnnealing(const unordered_set<int>& coveredVertices,
117     int maxIterations, double initialTemp, double coolingRate) {
118     // Inicializa a clique atual e a melhor clique com um vertice
119     // aleatorio nao coberto
120     vector<int> current = getRandomClique(coveredVertices);
121     vector<int> best = current;
122
123     // Inicializa a temperatura
124     double temp = initialTemp;
125
126     // Loop principal
127     for (int iteration = 0; iteration < maxIterations; ++iteration) {
128         // Obtem os vizinhos da clique atual que nao estao cobertos
129         vector<int> neighbors = getNeighbors(current, coveredVertices);

```

```

125
126 // Se nao ha vizinhos disponiveis, encerra o algoritmo
127 if (neighbors.empty()) break;
128
129 // Seleciona aleatoriamente um vizinho para explorar
130 int neighbor = neighbors[rand() % neighbors.size()];
131 vector<int> newClique = current;
132 newClique.push_back(neighbor);
133
134 // Verifica se a nova clique e de fato uma clique
135 if (isClique(newClique)) {
136     // Calcula a variacao de energia (tamanho da nova clique -
137     // tamanho da clique atual)
138     double deltaE = newClique.size() - current.size();
139
140     // Aceita a nova clique de acordo com uma probabilidade
141     // baseada na temperatura atual
142     if (deltaE > 0 || exp(deltaE / temp) > ((double) rand() /
143     RAND_MAX)) {
144         current = newClique; // Aceita a nova clique
145     }
146
147     // Atualiza a melhor clique se a atual for melhor
148     if (current.size() > best.size()) {
149         best = current;
150     }
151 }
152
153 // Reduz a temperatura de acordo com a taxa de resfriamento
154 temp *= coolingRate;
155
156 // Retorna a melhor clique encontrada
157 return best;
158 }
159
160 vector<int> tabuSearch(const unordered_set<int>& coveredVertices, int
161 maxIterations, int tabuTenure) {
162     // Inicializa a clique atual e a melhor clique com um vertice
163     // aleatorio
164     vector<int> current = getRandomClique(coveredVertices);
165     vector<int> best = current;
166
167     // Lista tabu para armazenar os cliques visitados recentemente
168     deque<vector<int>> tabuList;
169     tabuList.push_back(current); // Adiciona a clique atual na lista tabu
170
171     // Loop principal da Busca Tabu
172     for (int iteration = 0; iteration < maxIterations; ++iteration) {
173         // Obtem os vizinhos do clique atual que nao estao cobertos
174         vector<int> neighbors = getNeighbors(current, coveredVertices);
175
176         // Se nao ha vizinhos disponiveis, encerra o algoritmo
177         if (neighbors.empty()) break;
178
179         // Inicializa a melhor vizinhanca com a clique atual
180         vector<int> bestNeighbor = current;
181
182         // Itera sobre todos os vizinhos possiveis
183         for (int neighbor : neighbors) {
184             // Cria uma nova clique adicionando o vizinho na clique atual
185             vector<int> newClique = current;

```

```

182         newClique.push_back(neighbor);
183
184         // Verifica se a nova clique e um clique e se nao esta na
185         // lista tabu
186         if (isClique(newClique) && (find(tabuList.begin(), tabuList.
187         end(), newClique) == tabuList.end())) {
188             bestNeighbor = newClique; // Atualiza a melhor vizinhanca
189             encontrada
190             break; // Interrompe a busca apos encontrar uma vizinhanca
191             valida
192         }
193     }
194
195     // Se a melhor vizinhanca encontrada for melhor que a clique atual
196     if (bestNeighbor.size() > current.size()) {
197         current = bestNeighbor; // Atualiza o clique atual para a
198         // melhor vizinhanca encontrada
199
200         // Atualiza a melhor clique global, se o clique atual for
201         // melhor
202         if (current.size() > best.size()) {
203             best = current;
204         }
205
206         // Adiciona o clique atual na lista tabu
207         tabuList.push_back(current);
208
209         // Remove o clique mais antigo da lista tabu se a lista
210         // atingir o tamanho maximo permitido
211         if (tabuList.size() > tabuTenure) {
212             tabuList.pop_front();
213         }
214     } else {
215         break; // Encerra o loop se nao houver melhoria no clique
216         // atual
217     }
218 }
219
220 // Retorna o melhor clique encontrado
221 return best;
222 }
223
224 bool isSubset(const vector<int>& clique, const vector<vector<int>>&
225 cliques) {
226     for (const auto& existingClique : cliques) {
227         unordered_set<int> existingSet(existingClique.begin(),
228             existingClique.end());
229         bool isSubset = true;
230         for (int v : clique) {
231             if (existingSet.find(v) == existingSet.end()) {
232                 isSubset = false;
233                 break;
234             }
235         }
236         if (isSubset) {
237             return true;
238         }
239     }
240     return false;
241 }
242
243 vector<vector<int>> vertexCoverByCliques(int timeLimit, string strategy) {

```

```

234 // Inicializa o contador de tempo e a lista de cliques encontrados
235 auto start = chrono::steady_clock::now();
236 vector<vector<int>> cliques;
237 unordered_set<int> coveredVertices;
238 int maxIterations = 1000; // Numero maximo de iteracoes para as
    estrategias Simulated Annealing e Tabu Search
239 double initialTemp = 1000.0; // Temperatura inicial para o Simulated
    Annealing
240 double coolingRate = 0.95; // Taxa de resfriamento para o Simulated
    Annealing
241 int tabuTenure = 10; // Tamanho da lista tabu para o Tabu Search
242
243 // Loop principal para encontrar cobertura minima por cliques dentro
    do limite de tempo
244 while (coveredVertices.size() < V) {
245     vector<int> clique;
246
247     // Seleciona a estrategia de busca de acordo com o parametro '
        strategy'
248     if (strategy == "hc") // Hill-climbing
249         clique = hillClimbing(coveredVertices);
250     else if (strategy == "sa") // Simulated Annealing
251         clique = simulatedAnnealing(coveredVertices, maxIterations,
            initialTemp, coolingRate);
252     else if (strategy == "ts") // Tabu Search
253         clique = tabuSearch(coveredVertices, maxIterations, tabuTenure
            );
254
255     // Verifica se o clique encontrado nao e um subconjunto de cliques
        ja encontrados
256     if (!isSubset(clique, cliques)) {
257         cliques.push_back(clique); // Adiciona o clique aa lista de
            cliques encontrados
258
259         // Marca os vertices cobertos pela nova clique na cobertura
260         for (int v : clique) {
261             coveredVertices.insert(v);
262         }
263     }
264
265     // Verifica o tempo decorrido e encerra o loop se o limite de
        tempo definido for atingido
266     auto end = chrono::steady_clock::now();
267     auto duration = chrono::duration_cast<chrono::seconds>(end - start
        ).count();
268     if (duration >= timeLimit) {
269         break;
270     }
271 }
272
273 // Ordena os cliques encontrados do maior para o menor tamanho
274 sort(cliques.begin(), cliques.end(), [](const vector<int>& a, const
    vector<int>& b) {
275     return a.size() > b.size();
276 });
277
278 // Retorna a lista ordenada de cliques encontrados
279 return cliques;
280 }
281 };
282
283 int main(int argc, char* argv[]) {

```

```

284     if (argc != 4) {
285         cerr << "Uso: " << argv[0] << " <arquivoProblema> <tempoMaximo> <
            strategy>" << endl;
286         return 1;
287     }
288
289     string filename = argv[1];
290     int timetorun = atoi(argv[2]);
291     string strategy = argv[3];
292
293     ifstream file(filename);
294     if (!file.is_open()) {
295         cerr << "Erro ao abrir o arquivo " << filename << endl;
296         return 1;
297     }
298
299     string line;
300     unsigned int numVertices = 0;
301     unsigned int numEdges = 0;
302
303     string extension = ".grafo";
304     string::size_type i = filename.find(extension);
305     if (i != string::npos) { // extensao '.grafo'
306         getline(file, line);
307         stringstream ss(line);
308         ss >> numVertices;
309         ss >> numEdges;
310     } else { // extensao '.clq'
311         // Ler o cabecalho do arquivo e obter o numero de vertices e arestas
312         while (getline(file, line)) {
313             if (line.empty() || line[0] == 'p') {
314                 stringstream ss(line);
315                 string token;
316                 ss >> token; // descartar 'p'
317                 ss >> token; // esperado 'col'/'edge'
318                 ss >> numVertices;
319                 ss >> numEdges;
320                 break;
321             }
322         }
323     }
324     srand(time(0));
325
326     Graph graph(numVertices);
327
328     // Ler as arestas e adicionar ao grafo
329     if (i != string::npos) { // extensao '.grafo'
330         while (getline(file, line)) {
331             if (line.empty()) continue;
332             unsigned int from, to;
333             stringstream ss(line);
334             ss >> from >> to;
335             graph.addEdge(from, to); // Os vertices no arquivo comecam de 0.
336         }
337     } else {
338         while (getline(file, line)) {
339             if (line.empty() || line[0] != 'e') continue;
340             stringstream ss(line);
341             string token;
342             ss >> token; // descartar 'e'
343             unsigned int from, to;
344             ss >> from >> to;

```

```

345         graph.addEdge(from - 1, to - 1); // Os vertices no arquivo comecam
           de 1.
346     }
347 }
348 file.close();
349
350 vector<vector<int>> cliques;
351 cliques = graph.vertexCoverByCliques(timotorun, strategy);
352
353 cout << cliques.size() << endl;
354 for (const auto& clique : cliques) {
355     for (int v : clique) {
356         cout << v + 1 << " ";
357     }
358     cout << endl;
359 }
360
361 return 0;
362 }

```

Código 6: Implementação da Tarefa 2 - Problema do Mínimo Número de Cliques, com *Hill-climbing*, *Simulated Annealing* e *Tabu Search*.

Conforme as instruções de uso (seção 1.2.2), na execução do programa desenvolvido o usuário passa o nome do arquivo de instância, o tempo máximo de execução até a apresentação de uma saída e a estratégia desejada.

A função principal faz a leitura do arquivo de entrada, ignorando as linhas de comentário (iniciadas com o caractere 'c'), e obtém o número total de vértices e número total de arestas. Em seguida, cria um objeto do tipo *Graph*, e continua a leitura do arquivo de entrada, atribuindo ao grafo, para cada linha lida do arquivo, uma nova aresta.

A partir daí, inicia-se a contagem do tempo limite de execução do programa, o método do grafo para obter o menor número de cliques é chamado e o resultado é apresentado.

3 Estudo de Complexidade

Nesta seção são apresentados os estudos da complexidade do tempo de execução dos procedimentos implementados e do programa como um todo, considerando entradas de tamanho n (ou $|V|$, onde V é o número de vértices do grafo).

3.1 Análise de complexidade para o Problema do Clique Máximo

O método *isClique(const vector<int>& vertices)* verifica se os k vértices do vetor *vertices* formam um clique. Para cada par de vértices, uma busca é realizada nas listas de adjacência correspondentes. Portanto, no pior caso, k pode ser igual a $|V|$, resultando em uma complexidade na ordem de $O(|V|^2)$.

O método *getRandomClique()* seleciona um vértice aleatório em V , com custo constante, portanto sua complexidade de tempo é $O(1)$. O método *getNeighbors(const vector<int>& clique)* percorre a lista de adjacências de cada vértice v pertencente ao clique tomado como argumento. O número de listas pode chegar a $|V|$, se a instância de entrada representa um grafo completo. O tamanho da lista corresponde ao grau do vértice v , que no pior caso, pode ser igual a $|V| - 1$. Portanto, o método de busca pela vizinhança tem, no pior caso, ordem de complexidade $O(|V|^2)$.

Quanto aos métodos associados às estratégias de projeto dos algoritmos aproximados, têm-se:

- ***hillClimbing()***: No pior caso, o método é executado I vezes, onde I é o número máximo de iterações ou até o tempo máximo ser atingido. Dentro do *loop*, os métodos *getNeighbors* e *isClique* são chamados I vezes. Como visto, a complexidade de ambos pode chegar a $O(|V|^2)$. Portanto, a complexidade do método *hillClimbing* é da ordem de $O(I * |V|^2)$.
- ***simulatedAnnealing(double initialTemperature, double coolingRate)***: De forma similar ao método *hillClimbing*, o método é executado I vezes, com chamadas aos métodos *isClique* e *get-*

Neighbors e *probability*. Este último método tem custo $O(1)$. Portanto, a ordem de complexidade do método *simulatedAnnealing* é, no pior caso, igual a $O(I * |V|^2)$.

- ***tabuSearch(int maxIterations, int maxTabuSize)***: O número de vezes que o *loop* principal é executado é igual a *maxIterations* e os principais métodos chamados são os mesmos de *hillClimbing* e *simulatedAnnealing*. Adicionalmente, *tabuSearch* precisa lidar com a lista *tabuList*, cujas operações, no pior caso, têm custo linear (*clear*, *find*, etc)³. Portanto, a ordem de complexidade de tempo para o método *tabuSearch* é igual a $O(maxIterations * (|V| + |V|^2)) = O(I * |V|^2)$, onde $I = maxIterations$.

O programa como um todo tem, portanto, independentemente do método de solução aproximada escolhido (*Hill-climbing*, *Simulated Annealing* ou *Tabu Search*), ordem de complexidade, no pior caso, na ordem de $O(I * |V|^2)$.

3.2 Análise de complexidade para o Problema do Mínimo Número de Cliques

Na solução do problema do **Mínimo Número de Cliques**, parte dos métodos foram aproveitados da implementação da tarefa anterior. Portanto, já se sabe sua ordem de complexidade: *isClique* $\in O(|V|^2)$; *getRandomClique* $\in O(1)$; *getNeighbors* $\in O(|V|^2)$.

Os métodos associados às estratégias *Hill-climbing*, *Simulated Annealing* e *Tabu Search* também têm ordens de complexidade similares às da implementação para a *Tarefa 1*. No entanto, para a *Tarefa 2*, foi implementado o método que busca pela cobertura por cliques, *vertexCoverByCliques*.

Para analisar a ordem de complexidade do método *vertexCoverByCliques*, é preciso analisar mais detalhadamente o *loop* principal, que é executado até que todos os vértices estejam cobertos ou até o tempo limite de execução determinado.

A cada iteração, um dos métodos associados aos algoritmos aproximados é chamado. Todos eles têm a mesma ordem de complexidade, $O(|V| * d^2 * |k|^2)$, onde d é o grau médio dos vértices e k é o tamanho médio de um clique. Outro método que é chamado c vezes, onde c é o número de cliques atuais é o método *isSubset*, que verifica se um clique é subconjunto de algum outro já encontrado, evitando redundâncias. Seu custo é $O(c * k)$.

A inserção de um clique na cobertura ou a atualização da lista de cliques da cobertura têm custo $O(k)$. Portanto, até aqui, o custo do *loop* principal é da ordem de $O(|V| * (|V| * d^2 * k^2 + c * k + k)) = O(|V|^2 * d^2 * k^2 + |V| * c * k)$. No entanto, a lista de cliques da cobertura é ordenada, antes da apresentação dos resultados. Essa ordenação tem custo $O(c * \log c)$. Ainda assim, o termo de maior custo é $|V|^2 * d^2 * k^2$. Logo, a complexidade final do método é igual a $O(|V|^2 * d^2 * k^2)$, que é a ordem de complexidade do programa.

4 Testes

Esta seção descreve os experimentos realizados para os dois problemas abordados neste trabalho prático: **Tarefa 1 (Clique Máximo)** e **Tarefa 2 (Menor Número de Cliques)**. Para ambos os problemas, foram usados os mesmos arquivos de instância, passados como parâmetro via terminal.

A saída da **Tarefa 1** corresponde ao tamanho do clique máximo encontrado, seguido pelos vértices que o compõe, conforme o exemplo na Tabela 1. Para a **Tarefa 2**, a saída é o número de subcliques encontrados, seguido por cada um deles na linha abaixo, do maior para o menor, como o caso de teste da Tabela 2.

4.1 Testes para o Problema do Clique Máximo

O formato das instâncias de teste (**Entrada**) e das saídas desejadas (**Saída**) são apresentados no **Exemplo de caso de teste** (*brock200_4.clq*), conforme a especificação da **Tarefa 1** (Tabela 1).

Para cada estratégia de projeto de algoritmo aproximado (*Hill-climbing*, *Simulated Annealing* e *Tabu Search*), foram realizadas 20 execuções e os melhores resultados são apresentados a seguir, considerando um limite de tempo de execução igual a 1 minuto (60 segundos).

³unordered_set: https://cplusplus.com/reference/unordered_set/unordered_set/

Entrada	Saída
c FILE: brock200_4.b	17
p edge 200 13089	160 27 116 164 126 53 64 78 191 138 11 92 37 70 185 18 28
e 3 1	
e 4 3	
e 5 1	
e 5 3	
e 6 1	
e 6 2	
e 6 3	
e 6 5	
e 7 1	
...	

Tabela 1: Exemplo de entrada e saída esperada para a **Tarefa 1**.

Instância *brock200_4.clq*:

```
>task1-max-clique.exe brock200_4.clq 60 hc
Tempo de execucao: 0.0647 s.
13
63 1 3 5 6 22 31 33 53 72 85 95 98

>task1-max-clique.exe brock200_4.clq 60 sa
Tempo de execucao: 0.0198 s.
11
1 21 129 7 40 62 46 77 164 141 159

>task1-max-clique.exe brock200_4.clq 60 ts
Tempo de execucao: 0.0650 s.
13
111 2 7 9 16 17 42 46 60 73 108 120 131
```

Instância *c125.9.clq*:

```
>task1-max-clique.exe c125.9.clq 60 hc
Tempo de execucao: 0.8550 s.
27
65 1 2 4 5 7 8 9 10 11 16 17 19 24 35 36 39 41 45 47 52 57 63 96 98 104 107

>task1-max-clique.exe c125.9.clq 60 sa
Tempo de execucao: 0.0697 s.
26
53 26 21 41 48 46 117 101 120 67 17 59 114 34 19 31 90 82 8 69 54 74 123 18 80 45

>task1-max-clique.exe c125.9.clq 60 ts
Tempo de execucao: 0.7548 s.
27
75 1 3 4 5 6 7 8 9 10 11 13 17 18 19 21 39 41 45 47 63 73 91 92 98 107 119
```

Instância *C250.9.clq*:

```

>task1-max-clique.exe C250.9.clq 60 hc
Tempo de execucao: 6.0450 s.
34
143 1 2 5 7 9 12 13 14 21 23 29 33 36 39 41 50 55 63 66 77 83 108 115 118 125 136 148 178 181
197 214 218 234

>task1-max-clique.exe C250.9.clq 60 sa
Tempo de execucao: 0.1400 s.
28
5 114 166 104 89 100 147 109 182 155 107 74 120 183 131 110 31 235 248 125 87 105 113 176
222 115 121 41

>task1-max-clique.exe C250.9.clq 60 ts
Tempo de execucao: 5.9407 s.
34
156 1 2 5 6 9 12 13 14 21 23 29 32 41 44 50 55 63 65 66 77 83 99 108 119 125 138 148 159 180
195 197 201 218

```

Instância *d8f1b2660f576d7d.grafo*:

```

>task1-max-clique.exe d8f1b2660f576d7d.grafo 60 hc
Tempo de execucao: 6.1256 s.
22
79 1 3 4 8 13 18 26 32 44 48 60 131 180 210 231 258 286 486 622 943 975

>task1-max-clique.exe d8f1b2660f576d7d.grafo 60 sa
Tempo de execucao: 0.1800 s.
16
925 638 347 461 673 266 171 796 585 198 814 687 895 563 677 56

>task1-max-clique.exe d8f1b2660f576d7d.grafo 60 ts
Tempo de execucao: 6.4204 s.
22
150 1 3 7 12 16 18 29 37 39 63 109 112 141 157 282 348 360 398 472 622 648

```

4.2 Testes para o Problema do Mínimo Número de Cliques

As instâncias de teste para o problema de encontrar o mínimo número de cliques em grafos não direcionados foram as mesmas da tarefa 1. O ***Exemplo de caso de teste***, para a instância *C125.9.clq*, pode ser visto na Tabela 2.

A seguir, temos as saídas com os melhores resultados para as principais instâncias de teste, considerando um limite de tempo de execução igual a 1 minuto (60 segundos) e 10 execuções para cada estratégia. Parte da saída foi suprimida para caber em uma linha.

Instância *brock200_4.clq*:

Entrada	Saída
c FILE: C125.9.clq	7 // Parte da saída foi suprimida para caber em uma linha
p col 125 6963	24 15 18 21 29 31 35 44 49 54 64 68 72 74 82 83 92 ...
e 2 1	23 1 2 10 11 27 32 46 58 60 61 73 76 78 81 85 88 94 ...
e 3 1	21 4 12 14 16 20 26 39 45 50 51 55 57 62 65 66 75 86 ...
e 4 1	22 6 8 9 13 19 23 30 33 37 38 42 47 52 53 56 59 80 87 ...
e 4 2	18 3 7 28 40 41 63 67 69 71 84 91 93 97 98 99 104 106 ...
e 4 3	14 5 17 24 25 34 48 70 77 79 89 115 122 123 125
e 5 1	3 22 43 96
e 5 2	
e 5 3	
e 5 4	
...	

Tabela 2: Exemplo de entrada e saída para a **Tarefa 2**.

```
>task2-min-cliques.exe brock200_4.clq 60 hc
25
103 1 5 6 16 22 24 26 40 53 61 91 95
36 2 7 9 21 23 46 49 65 92 108 132
190 3 4 8 13 18 25 27 73 78 180
188 15 28 33 45 50 59 67 152 157 197
112 37 38 71 79 93 129 139 145 170 198
193 10 11 12 14 29 43 90 121 182
56 17 30 32 42 48 55 63 116 191
86 19 47 52 57 69 72 81 113 127
125 44 75 76 77 89 101 110 137 138
131 34 60 68 82 111 128 133 165
186 87 94 106 109 142 153 183 189
115 58 66 70 117 120 123 179 195
96 39 51 54 62 64 85 141 151
175 80 83 88 114 126 146 161
178 84 97 98 119 143 144 156
118 20 31 35 41 74 134
181 124 136 148 155 160 185
192 99 104 107 150 164 174
166 102 122 140 163 167 169
184 100 130 147 196
177 105 149 154 168
187 158 171 199
172 135 162 173
159 176
200 194
```

```
>task2-min-cliques.exe brock200_4.clq 60 sa
25
196 40 153 91 141 53 162 140 31 183 168
170 177 146 190 45 189 144 115 26 43 95
38 70 8 117 76 102 65 161 12 139 137
192 7 108 72 155 2 143 132 106 151 97
187 134 37 57 156 174 178 100 41 71
54 6 175 68 128 1 33 160 16 3
154 61 130 22 125 157 119 103 121 90
194 84 77 92 148 69 21 145 186 74
193 23 138 4 131 64 55 25 34 78
15 188 113 152 85 51 124 101 197
79 165 129 27 109 179 75 180 135
60 5 52 39 63 184 47 112 142
116 19 18 114 30 191 28 169 56
173 163 176 107 96 59 159 111 123
158 118 32 17 120 149 198 67
127 182 171 94 29 199 105 87
49 93 81 110 11 136 9
62 80 122 46 86 133 14
98 172 147 200 42 166 104
181 48 50 44 24 13 10
185 89 99 73 58 167
83 164 36 88 126
66 35 82 195
150
20
```

```
>task2-min-cliques.exe brock200_4.clq 60 ts
25
141 4 7 17 25 38 46 55 63 64 85 191
138 12 14 29 35 40 72 90 95 137 143 193
142 2 8 9 13 31 53 61 65 83 168
101 1 3 6 15 26 27 80 98 124 162
140 30 44 48 52 75 93 97 127 186 189
155 10 11 23 33 41 62 115 119 181 200
150 66 73 74 79 99 107 118 134 156
37 5 16 19 24 43 57 91 103 148
152 22 36 42 51 56 71 92 177
165 54 59 60 67 68 104 117 179
132 82 96 102 122 125 144 187 197
106 18 20 21 28 32 34 39
111 78 86 116 131 151 169 176
170 89 114 120 123 133 167 198
199 45 49 50 84 108 128 160
113 47 58 69 70 81 135 136
145 77 94 109 110 121 158
196 76 87 88 126 164 171
149 139 153 154 157 180 195
183 100 112 129 159 184
173 163 166 174 178
188 105 130 182
146 161 175 190
147 172 185
192 194
```

Instância *c125.9.clq*:

```
>task2-min-cliques.exe c125.9.clq 60 hc
7
67 3 4 12 13 20 23 24 26 28 29 30 39 40 45 49 51 55 58 60 61 86 99 101 110 118
76 1 2 5 6 7 8 9 10 11 17 18 19 21 36 41 47 48 63 92 93 94 98 107
83 14 15 16 22 27 33 34 43 52 54 56 59 78 81 82 85 88 95 104 111 117 123
90 25 31 32 37 46 53 65 69 70 71 74 79 80 96 102 106 113 114 124 125
72 35 38 42 44 50 62 64 66 73 77 87 91 121
120 57 75 84 89 100 105 109 115 116
108 68 97 103 112 119 122
```

```
>task2-min-cliques.exe c125.9.clq 60 sa
8
14 9 61 21 125 77 54 82 75 1 17 58 48 115 104 123 44 49 29 31 45 121 117 5 114 92 70 11
25 111 60 113 85 107 119 101 116 39 10 100 79 94 46 103 110 2 22 80 7 32 74 12 6
97 67 50 8 37 3 33 68 63 69 18 72 64 38 59 13 20 96 47 53
55 106 23 84 124 62 71 91 66 57 65 40 109 35 78 4 51 86 98 28
99 108 24 76 27 81 118 19 56 122 88 15 52 43
73 16 102 105 34 87 30 26
93 95 112 36 120 42 41
83 89 90
```

```
>task2-min-cliques.exe c125.9.clq 60 ts
8
106 2 4 5 6 7 8 9 10 11 16 17 18 24 35 36 37 45 47 49 63 67 92 96 117 123
113 1 3 13 15 21 23 28 29 33 39 40 44 52 54 55 58 60 93 99 101 103 111 114 125
115 12 14 19 22 25 26 30 32 41 46 48 53 59 69 74 77 80 86 110 112 118
62 42 64 65 66 70 71 73 76 78 87 91 98 102 104 109
116 20 31 34 38 43 50 56 57 68 72 83 85 95
88 27 51 61 75 82 84 100 105 119 124
81 79 89 90 94 107 120
97 108 121 122
```

Instância *C250.9.clq*:

```
>task2-min-cliques.exe C250.9.clq 60 hc
12
210 1 5 6 8 12 13 14 16 18 21 29 30 32 36 41 44 50 63 66 75 84 99 108 115 152 160 ...
246 2 3 4 10 11 19 22 23 24 27 39 40 42 47 52 55 69 85 86 90 91 94 102 112 128 153 ...
58 17 20 26 31 34 37 43 49 56 60 61 70 71 89 92 96 105 136 141 143 150 157 165 192 201 ...
164 7 9 15 25 33 35 45 46 62 64 65 76 78 80 103 116 121 138 144 145 156 159 178 184 ...
57 38 48 51 53 54 67 72 77 79 81 83 87 104 106 107 113 114 125 133 139 140 176 222 ...
59 28 68 73 88 95 97 98 100 109 111 117 118 120 123 130 135 142 148 149 196 200 211 ...
126 74 82 93 101 110 122 124 129 134 137 147 161 166 173 180 190 203 206 225 235 241
204 119 127 131 151 167 169 171 172 174 175 186 189 191 207 214 231 237
244 132 146 163 168 170 179 182 185 188 193 208 215 224 234 236
249 154 155 158 194 209 213 227 239 240 243 245 247
220 162 216 219
199 217
```

```
>task2-min-cliques.exe C250.9.clq 60 sa
12
248 64 147 128 243 162 246 59 166 176 238 65 235 125 26 155 126 151 141 89 61 177 ...
231 53 83 135 237 226 144 189 115 87 107 63 145 30 42 76 18 173 93 183 195 66 233 159 ...
38 163 146 120 188 16 99 190 214 33 161 91 229 213 208 74 132 97 212 174 218 12 5 46 ...
88 156 118 150 209 81 211 153 165 31 133 157 241 201 80 103 51 104 168 139 100 82 94 ...
149 221 234 171 167 143 73 41 67 122 90 102 244 196 230 142 152 131 48 36 84 25 13 158 207
47 21 137 6 228 58 17 123 79 22 85 68 160 227 50 70 4 37 60 194 86 216 8
180 15 247 187 95 198 71 124 43 35 1 127 164 169 75 45 197 242 116 9
148 184 98 44 220 193 105 69 78 34 175 62 225 108 245 40 170 129 202 134
136 77 101 2 203 154 239 109 250 206 179 106 39 54 219
56 182 112 232 117 24 7 19 49 14 178 10 181 11 27
200 222 210 172 130 121 110 55
217 236 223 192 185 249
```

```
>task2-min-cliques.exe C250.9.clq 60 ts
12
142 1 2 5 6 8 12 13 14 21 23 29 30 32 36 41 44 50 55 61 63 77 83 99 108 115 152 160 ...
67 9 17 31 34 37 43 56 59 60 71 73 87 89 94 124 126 127 136 138 141 150 161 174 180 ...
244 3 4 10 11 16 18 19 24 27 42 47 49 52 65 68 69 75 84 102 109 110 128 134 153 158 ...
165 7 15 20 22 25 26 28 33 35 39 45 46 62 64 76 78 86 90 117 129 132 156 162 170 194 ...
101 38 40 48 51 53 57 66 85 88 92 95 103 104 105 107 113 114 116 130 139 140 166 172 203 204
233 54 70 72 74 79 91 96 97 98 106 111 120 123 135 144 149 157 159 167 182 183 202 219 250
249 58 81 93 119 121 122 125 133 145 151 179 187 191 192 206 226 227 232 235 236 245
188 80 82 100 112 118 131 146 154 168 169 173 178 190 195 209 210 211 246
241 137 143 147 148 155 163 164 171 175 176 177 207 220 237 240
243 184 185 186 189 198 205 213 215 216 221
222 193 208 223 224 228 231 234 239
229 200 217 225
```

Instância *d8f1b2660f576d7d.grafo*:

```
>task1-max-clique.exe d8f1b2660f576d7d.grafo 60 hc
10
367 5 9 11 18 20 28 60 68 69 113 185 210 214 234 319 394 460 506 729 935
49 3 8 17 22 23 35 52 88 93 119 128 151 224 270 306 342 361 519 751
36 12 13 25 38 39 47 48 51 55 150 168 230 262 283 303 334 586 650 724
430 10 24 30 31 37 43 59 71 82 131 164 219 258 360 493 622 670 740 977
961 21 26 41 50 53 57 70 76 83 124 140 176 187 339 372 625 690 810
264 33 56 58 80 98 102 112 116 134 147 165 226 233 359 428 593 783 907
990 45 66 78 94 97 111 138 153 182 218 253 280 402 479 488 512 754 847
820 1 2 4 6 7 16 19 27 81 84 92 95 109 169 376 398 887
456 14 15 29 32 34 40 44 89 96 114 158 193 244 287 323 388 752
879 42 46 62 65 72 75 79 85 90 148 171 186 220 490 707 735
```

```
>task1-max-clique.exe d8f1b2660f576d7d.grafo 60 sa
16
194 829 83 396 754 721 814 381 302 801 151 935 178 773 926 507 978 774 235 77 631
769 453 50 825 67 339 877 884 585 931 215 489 224 974 487 362 71 154 40 505
764 549 853 126 393 995 820 574 332 43 875 168 426 759 986 484 976 568 93
990 522 958 810 980 572 358 903 783 368 895 892 150 24 233 473 607 890 440
372 94 709 982 97 179 907 718 419 70 499 831 893 786 546 134 688 775
22 534 429 981 262 106 271 885 432 121 740 593 658 237 170 259 361 739
1000 353 766 402 670 399 501 329 857 644 521 540 120 703 625 242 930 517
297 613 343 27 750 971 674 845 220 174 525 351 210 231 539 560 819 795
793 404 462 389 138 476 434 598 441 830 647 344 821 859 74 959 155 397
621 837 591 228 516 855 643 692 291 51 350 632 48 532 506 991 781 733
578 870 307 374 941 101 363 300 544 869 251 175 656 636 660 375 565 318
156 804 81 292 75 102 622 654 811 668 66 945 11 2 847 131 7 159
757 377 245 662 882 889 425 751 535 395 21 298 977 345 371 650 176 983
900 23 646 49 682 369 438 289 784 942 752 587 500 133 161 200 611 257
973 69 891 880 58 681 125 33 123 428 206 333 38 608 835 996 311 167
449 265 166 282 376 553 88 648 791 59 244 31 269 642 165 284 699
```

```
>task1-max-clique.exe d8f1b2660f576d7d.grafo 60 ts
67
795 136 144 165 195 196 202 221 227 250 301 357 378 435 457 464 540 630 681 711 969
760 92 107 128 132 137 149 159 179 234 287 300 305 415 446 465 539 811 812 827 900
729 60 64 76 80 83 93 95 104 115 162 182 266 270 306 344 403 589 694 763
121 66 75 90 98 106 114 116 127 156 166 169 173 193 200 229 433 529 730 910
562 1 2 3 4 7 12 16 35 36 43 61 109 141 157 172 398 680 996
833 283 290 308 312 313 324 336 340 347 379 391 455 635 718 723 735 771 858
278 21 31 37 46 51 54 62 71 82 101 244 268 281 714 790 808 955 961
208 33 53 55 59 63 74 81 105 135 143 181 205 422 425 471 674 751 823
701 380 397 400 434 438 462 463 507 514 516 569 622 664 720 804 830 873 953
779 103 119 120 122 133 175 177 223 228 248 323 332 382 537 592 848 904 929
725 158 176 178 183 188 215 275 320 420 466 519 534 587 816 851 863 892 944
792 126 138 150 155 168 170 222 247 255 352 374 429 505 703 726 769 819 994
364 129 130 139 189 190 198 199 201 219 263 285 294 296 311 333 512 758 995
335 5 6 9 11 20 27 28 57 111 164 174 224 240 251 316 386 467 878
262 213 242 249 252 254 260 295 314 385 389 474 538 733 740 836 876 902
915 337 375 376 401 416 428 447 470 476 499 501 607 673 679 800 972 991
924 192 194 220 225 267 282 284 318 328 393 430 488 527 563 728 844 860
843 206 216 233 235 239 243 302 304 309 345 348 479 500 511 669 687 784 665 411 414 417 436
478 510 555 558 573 577 623 627 705 803 818 939 951
258 73 87 91 94 97 102 140 184 191 211 259 349 368 373 439 460 834
99 45 56 58 70 77 78 86 112 124 153 217 230 253 359 363 399 530
976 8 13 18 25 26 38 48 52 85 161 185 272 307 317 370 413 543
180 17 19 22 30 40 41 50 84 88 142 207 291 292 339 486 506 690
608 39 100 110 117 123 125 145 151 171 187 214 381 406 531 544 581
748 113 118 134 146 147 152 154 167 212 241 261 264 619 632 660 693
911 271 274 288 303 322 327 330 360 392 437 508 559 625 645 767 936
594 14 15 32 34 44 67 72 89 96 163 238 265 286 334 547 643
597 277 289 326 338 341 346 371 383 440 444 454 517 521 618 778 971
874 355 356 369 387 418 426 427 494 524 548 568 570 686 717 917 990
65 10 23 24 29 42 47 49 68 69 79 108 131 148 342 599 719
591 160 186 197 203 204 210 218 231 232 310 325 564 566 797 966
959 513 528 557 600 614 640 647 671 699 739 810 847 859 907 992
891 451 472 482 492 503 509 522 595 659 677 696 700 842 862 950
901 404 449 450 483 485 493 496 565 571 574 580 605 662 708 882
661 321 331 350 354 358 377 394 487 515 546 585 634 813 869 998
651 237 246 276 280 293 299 315 329 343 366 402 432 452 655 698
650 407 412 421 453 468 495 497 518 545 549 593 752 886 952
541 442 459 461 477 489 520 532 536 606 646 712 766 942 964
985 236 256 273 279 297 298 353 372 410 423 431 475 579 648
554 443 473 480 523 526 561 576 586 609 654 656 704 732 967
828 484 533 550 553 603 612 637 638 736 746 750 785 798 916
388 209 226 245 257 269 319 351 361 367 409 445 610 658 765
846 551 582 588 629 631 675 692 697 702 773 791 831 986
960 560 601 604 613 615 621 672 753 761 768 806 815 893
822 481 491 542 567 590 598 602 611 636 644 667 724 856
...
```

5 Análises e Considerações

A Tabela 3 apresenta alguns números relacionados à execução das instâncias de teste com as diferentes estratégias de projeto de algoritmos aproximados para o *Problema do Clique Máximo (PCM)* e para o *Problema do Menor Número de Cliques (PMNC)*.

Tabela 3: Resultados dos testes: Comparativo entre estratégias.

instância	# vértices	# arestas	PCM ¹	PCM	PMNC ¹	PMNC
brock200_4.clq (hc)	200	13089	11	13	22	25
brock200_4.clq (sa)	200	13089	11	11	22	25
brock200_4.clq (ts)	200	13089	11	13	22	25
c125.9.clq (hc)	125	6963	24	27	7	7
c125.9.clq (sa)	125	6963	24	26	7	8
c125.9.clq (ts)	125	6963	24	27	7	8
C250.9.clq (hc)	250	27984	31	34	11	12
C250.9.clq (sa)	250	27984	31	28	11	12
C250.9.clq (ts)	250	27984	31	34	11	12
d8f1b2660f576d7d.grafo (hc)	1000	369662	-	22	-	10
d8f1b2660f576d7d.grafo (sa)	1000	369662	-	16	-	16
d8f1b2660f576d7d.grafo (ts)	1000	369662	-	22	-	67

¹Valores de Referência fornecidos pelo Professor.

Para o PCM, o *Simulated Annealing* foi a estratégia que chegou a resultados mais rapidamente, no entanto, obteve cliques máximos menores, sinalizando sempre encontrar *soluções ótimas locais*.

As estratégias *Hill-climbing* e *Tabu Search* tiveram desempenho similar entre si, tanto em tempo de execução, quanto em relação ao tamanho do clique máximo encontrado.

Todas as estratégias tiveram resultados muito próximos aos valores de referência fornecidos pelo Professor, para algumas instâncias (*brock200_4.clq*: *maxclique* = 11; *c125.9.clq*: *maxclique* = 24; *C250.9.clq*: *maxclique* = 31).

As implementações para o PMNC também tiveram resultados parecidos, muito próximos dos valores de referência (*brock200_4.clq*: *subset* = 22; *c125.9.clq*: *subset* = 7; *C250.9.clq*: *subset* = 11).

Destaca-se que, dentro do limite máximo de tempo de execução (60 segundos), apenas a estratégia *Tabu Search* foi capaz de obter um resultado com a cobertura completa dos vértices da instância com tamanho igual a 1000 vértices (*d8f1b2660f576d7d.grafo*).

Referências

- [1] Alvaro Leandro Cavalcante Carneiro. Algoritmos de otimização: Hill climbing e simulated annealing, 2020. <https://medium.com/data-hackers/algoritmos-de-otimiza%C3%A7%C3%A3o-hill-climbing-e-simulated-annealing-3803061f66f0> [Accessed: 2024-07-09].
- [2] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms - Third Edition*. The MIT Press, Cambridge, Massachusetts, 2009.
- [3] Wen-Lian Hsu and George L. Nemhauser. Algorithms for minimum covering by cliques and maximum clique in claw-free perfect graphs. *Discrete Mathematics*, 37(2):181–191, 1981.
- [4] Donald L. Kreher and Douglas R. Stinson. *Combinatorial algorithms: generation, enumeration, and search*. CRC Press LLC, Boca Raton, Florida, 1999.